FACULTY OF ENGINEERING AND NATURAL SCIENCES

CAPSTONE FINAL REPORT

SMART BOTTLE WASTE-TO-FILAMENT SYSTEM (RECYPRINT)

Abubakr Suliman (Mechatronics Engineering)

Eda Yaygılı (Software Engineering)

Mohammed Younis (Mechatronics Engineering)

Selen Göğüş (Software Engineering)

Şevval Naz Savaş (Software Engineering)

Assist Prof. Amir Navidfar (Mechatronics Engineering)
Assist. Prof. Sama Habibi (Software Engineering)

June, 2026

STUDENT DECLARATION

By submitting this report, as partial fulfilment of the requirements of the Capstone Design Project course, each member of the project group certifies that:

they have given credit to and declared (by citation), any work that is not their own;

they have not received unpermitted aid for the project design, construction, report writing or presentation;

they have not falsely assigned credit for work to another student in the group and that each member's contribution is properly noted.

DECLARATION OF AI USAGE

This project report was developed with the assistance of generative AI tools (e.g., ChatGPT, Copilot, Gemini, Claude, etc.). The use of AI in this work was limited to the following activities:

- Brainstorming and idea generation
- Literature search and summarization
- Grammar and language editing
- Code generation and debugging assistance
- Data analysis support
- Other (please specify): _____

Validation and Authorship I certify that:

I have personally verified all facts, citations, and data points provided by the AI.

I have not simply copied and pasted AI outputs; all final text is my own writing or a significant rewrite of AI suggestions.

I accept full responsibility for the integrity and accuracy of this report.

Project Team

Department 1 : Mechatronics Engineering

Member 1 : Abubakr Suliman

Member 2 : Mohammed M. A. Younis

Advisor : Assist Prof. Amir Navidfar

Department 2 : Software Engineering

Member 1 : Şevval Naz Savaş

Member 2 : Eda Yaygılı

Member 3 : Selen Göğüş

Advisor : Assist. Prof. Sama Habibi

ABSTRACT

Problem Statement: The increasing accumulation of PET plastic waste necessitates localized, efficient recycling solutions. Transforming waste plastic bottles into usable 3D printer filament offers a highly practical approach. However, achieving consistent filament quality requires precise thermal and mechanical control, tightly integrated with a reliable real-time monitoring and data logging system to ensure operational safety and process efficiency.

Methodology Used: The RecyPrint project engineered a fully integrated, end-to-end IoT-based mechatronic system. The physical extrusion process is governed by a custom "RecyPrint OS" deployed on an ESP32 microcontroller, which handles localized high-speed PID thermal regulation and motor control. To ensure high-performance communication, a tethered Python MQTT bridge was developed to translate physical hardware telemetry into strict JSON payloads. The software architecture utilizes a Flask backend, a Neon PostgreSQL database for cloud-hosted logging , and a newly implemented local SQLite fallback to guarantee data integrity during offline or network-unstable scenarios. A web-based dashboard serves as the central interface for the entire operation.

Key Results and Findings: Full system integration was successfully achieved. The physical machine now actively transmits live, accurate sensor data—including real-time extrusion temperature and filament diameter (tracked via an SS49E Hall Effect sensor)—directly to the web dashboard. Early critical challenges, such as electromagnetic interference (EMI) causing system crashes, were successfully resolved by optimizing PWM frequencies. Most notably, bidirectional communication was fully validated: the system allows complete remote control, enabling users to adjust thermal setpoints and safely start or stop the extrusion motor directly from the live dashboard.

Conclusions: The successful transition from simulated software data to active physical hardware integration confirms the viability and stability of the RecyPrint system. The architectural pivot to a tethered MQTT bridge, combined with the dual-database approach (Neon cloud and SQLite local), resulted in a highly stable, uninterrupted, and easily controllable plastic extrusion process.

Contributions of the Project: RecyPrint contributes a functional, modular blueprint for small-scale PET recycling. It successfully demonstrates how edge-control mechatronics can be seamlessly bridged with scalable digital infrastructure, providing a robust interface for both real-time process intervention and comprehensive historical reporting.

Key Words: PET Recycling, IoT Monitoring, Mechatronics Integration, Edge Control, MQTT Dashboard

TABLE OF CONTENTS

Right-click here and select 'Update Field' to generate Table of Contents

LIST OF TABLES

Right-click here and select 'Update Field' to generate List of Tables

LIST OF FIGURES

Right-click here and select 'Update Field' to generate List of Figures

LIST OF ABBREVIATIONS

[List abbreviations used in the report.]

1. Introduction

1.1 Background

The rapid expansion of additive manufacturing, commonly known as 3D printing, has driven a significant global demand for plastic filament. Simultaneously, the continuous accumulation of Polyethylene Terephthalate (PET) plastic waste, primarily from single-use beverage bottles, presents a critical environmental challenge. The intersection of these two areas provides a unique opportunity for localized, circular manufacturing. By transforming waste plastic directly into usable 3D printer filament, it is possible to create sustainable, closed-loop material life cycles that reduce both waste and the need for newly manufactured plastics.

This project falls within the intersecting domains of the Internet of Things (IoT), Mechatronics, and Software Engineering. Historically, plastic extrusion and recycling have been large-scale, opaque industrial processes. Translating this capability to a localized, smart-system level requires a complex, multidisciplinary approach. At the physical level, it involves mechatronics for precise physical actuation, requiring rigorous thermal regulation, mechanical spooling, and continuous diameter sensing.

In the broader technological landscape, the project operates within the domain of Edge Computing and Smart Manufacturing. Modern production environments demand that physical machines do not operate in isolation. To achieve reliable filament quality and operational safety, this project leverages an IoT architecture utilizing lightweight telemetry protocol (MQTT), cloud-hosted database integration, and real-time web visualization. By bridging edge-control firmware with scalable backend digital infrastructure, the project transforms a raw mechanical recycling process into a smart, data-driven, and remotely controllable production system.

1.2 Problem Statement

The global reliance on single-use plastics, particularly Polyethylene Terephthalate (PET) beverage bottles, has led to a compounding environmental crisis. While large-scale recycling infrastructure exists, it is often energy-intensive and geographically centralized, leaving a significant portion of plastic waste unprocessed in local communities. Simultaneously, the rapid growth of the 3D printing industry has created a continuous demand for plastic filament. Commercially produced filament not only carries a substantial financial cost for makers, educational institutions, and hobbyists, but it also possesses a hidden environmental cost due to the manufacturing and global shipping processes.

The core problem this project addresses is the lack of accessible, smart, and localized recycling solutions that can directly bridge the gap between everyday plastic waste and functional manufacturing materials. While some small-scale or DIY extrusion machines exist, they typically lack the precision required to produce consistent, high-quality 1.75 mm filament. Without strict thermal regulation, mechanical stability, and real-time data monitoring, these rudimentary systems frequently suffer from filament snapping, inconsistent diameters, and hardware failures caused by overheating. This unreliability often leads to wasted effort, ruined materials, and potential safety hazards.

This problem affects local communities striving for better waste management, 3D printing practitioners seeking sustainable and cost-effective materials, and the broader environment burdened by plastic pollution. There is a critical need for a fully integrated, mechatronic system that not only

recycles PET waste at a localized level but also utilizes a resource-efficient, real-time digital monitoring infrastructure to ensure operational safety, prevent material loss, and guarantee consistent filament quality.

1.3 Project Objectives

The primary objective of the RecyPrint project is to engineer a fully integrated mechatronic system that recycles PET plastic bottles into usable 3D printer filament. This requires a stable physical extrusion machine paired with a cloud-connected, real-time monitoring and control dashboard to track temperature, motor speed, and filament diameter.

Backend & API Development: To develop a lightweight, scalable web backend using the Flask framework to serve the user interface and handle API requests.

Hybrid Database Management: To integrate a cloud-hosted AWS Neon PostgreSQL database for historical logging, coupled with a local SQLite caching mechanism to buffer sensor data locally and synchronize it when internet connectivity is lost.

Real-Time Communication: To establish low-latency, secure data pipelines using the MQTT protocol (WSS) to receive and process telemetry data, such as nozzle temperature and filament diameter.

Remote Control Interface: To implement a responsive dashboard interface that allows users to remotely monitor system states, start and stop the extruder motor, and transmit calibration commands (such as sensor taring) back to the hardware.

Modified Objectives: Although the core project objectives remained unchanged, critical implementation-level adjustments were made to ensure system stability and practicality

- **Database Strategy:** The primary database was migrated from a local Docker environment to cloud-hosted Neon PostgreSQL to improve system portability. Additionally, to prevent telemetry data loss during network outages, a local SQLite fallback (local_cache.db) was implemented as a robust offline-buffering solution.

1.4 Scope and Limitations

Scope of the Project: The software scope of the RecyPrint project encompasses the complete end-to-end digital infrastructure required to securely monitor, log, and remotely control the plastic extrusion process. The specific inclusions are:

- **Dashboard Interface:** Development of a real-time web application to visualize live process metrics and historical trends, including filament diameter, motor speed, and extrusion temperature.
- **Communication Bridge:** Establishment of a bidirectional communication layer using a tethered Python MQTT bridge to parse and route strict JSON telemetry between the local hardware and the cloud.

- **Remote Command Execution:** Implementation of bidirectional control logic, allowing users to remotely start and stop the automated extrusion process (which triggers target temperature setpoints and autonomous motor speed regulation) directly from the web interface.

Explicit Exclusions: Focusing strictly on the digital and software domain, the following areas are explicitly excluded from this system boundary: the mechanical design and 3D printing of the spooling mechanism, the material science analysis of recycled PET polymers, the physical wiring of the ESP32 microcontroller, and the internal tuning of the hardware-level electrical PID loops.

Technical Constraints

- **Network Dependency for Remote Operations:** While the newly integrated SQLite fallback successfully prevents historical data loss during internet outages, live dashboard updates and remote motor/temperature control remain strictly dependent on a stable internet connection and the uptime of the external MQTT broker.
- **Asynchronous Latency:** Achieving safe bidirectional control requires minimizing communication latency. The software is constrained by the speed at which the Python bridge can read serial outputs, format them into JSON, and publish them to the cloud without dropping packets or bottlenecking the host computer's processing threads.

Realistic Constraints

- **Economic:** To maintain a feasible project budget, the digital infrastructure relies entirely on open-source frameworks (Flask, Python, SQLite) and free-tier cloud hosting (Neon PostgreSQL, EMQX Broker). This imposes hard constraints on maximum database storage capacity, bandwidth limits, and concurrent connection allowances.
- **Environmental & Resource Efficiency:** The software architecture is designed with digital sustainability in mind. By formatting telemetry into strict, lightweight JSON payloads and utilizing an offline-first data logging strategy when applicable, the system minimizes redundant continuous cloud polling, thereby reducing the digital energy footprint and server load of the monitoring process.
- **Health & Safety:** The digital system is constrained by the absolute necessity of operational safety. The software must process incoming telemetry instantly to detect thermal runaways (e.g., temperatures exceeding safe bounds) and ensure that remote "Emergency Stop" commands sent from the dashboard are prioritized and routed with zero delay to prevent physical hazards.
- **Maintainability & Ethics:** The system relies on transparent, well-documented code repositories. A core constraint is ensuring the software remains open and maintainable, avoiding proprietary black-box algorithms so future developers can easily audit the safety logic or expand the architecture.

1.5 Report Organization

[Provide a brief outline of the report structure, describing what each section covers.]

2. Literature Review / State of the Art

[Provide a comprehensive review of related work, including academic papers, existing products, technologies, and standards relevant to the project. Identify gaps in existing solutions that your project addresses. Update and expand the literature review from the proposal with any new sources discovered during the project. Use IEEE citation format.]

The integration of Internet of Things (IoT) technologies into industrial systems has significantly transformed modern manufacturing processes. IoT enables real-time monitoring, remote control, and data-driven optimization by connecting sensors, controllers, and cloud platforms through communication networks. These capabilities are particularly valuable in small-scale and distributed production environments where continuous supervision and flexibility are required.

Among various communication protocols, MQTT (Message Queuing Telemetry Transport) has become one of the most widely adopted standards in IoT-based systems. MQTT follows a lightweight publish-subscribe architecture, making it suitable for low-bandwidth and resource-constrained environments. Its ability to provide reliable and asynchronous communication makes it highly suitable for real-time monitoring and control applications in industrial automation [1].

In parallel, sustainability-driven research has focused on recycling plastic waste, particularly PET (Polyethylene Terephthalate), into reusable materials such as filament for additive manufacturing. Several studies highlight the potential of converting PET waste into 3D printing filament as a cost-effective and environmentally friendly solution [2]. These systems aim to reduce plastic waste while enabling decentralized manufacturing.

However, most existing PET recycling and extrusion systems primarily focus on mechanical and material aspects, such as extrusion efficiency and filament quality. They often lack advanced digital infrastructure for real-time monitoring, remote accessibility, and data logging. In addition, many low-cost implementations do not provide reliable communication mechanisms or fail to address issues such as network interruptions and data loss.

Existing commercial and academic solutions also tend to operate as isolated systems, with limited integration between hardware and cloud-based services. This creates challenges in scalability, accessibility, and system reliability, especially in remote or resource-constrained environments.

To address these gaps, the RecyPrint project introduces a software-oriented IoT architecture that combines real-time monitoring, remote control, and robust data management. The system integrates MQTT-based communication, a web-based dashboard for user interaction, and a hybrid database structure using Neon PostgreSQL (cloud) and SQLite (local fallback). This approach enhances system reliability by ensuring continuous data logging even in the absence of network connectivity.

Compared to existing extrusion and recycling systems, the proposed RecyPrint platform provides a more comprehensive digital solution. While traditional and low-cost systems primarily focus on mechanical extrusion, they lack real-time monitoring, remote accessibility, and integrated cloud-based data management.

In contrast, the RecyPrint system enables real-time data visualization, remote control through a web-based interface, and reliable data storage using a hybrid cloud-local database architecture. Additionally, the use of MQTT ensures efficient and scalable communication between system components.

This comparison highlights that the main contribution of the project lies in the development of a robust and integrated software infrastructure rather than improvements in mechanical design.

3. Methodology and Technical Approach

3.1 Overall Approach

The overall methodology for the RecyPrint software infrastructure is rooted in modular system design and iterative software prototyping. Because the digital backend required seamless integration with a custom mechatronic edge device, an IoT-centric software architecture was adopted. The approach prioritized decoupling the physical hardware controls from the cloud communication layers to maximize system stability and development flexibility.

Engineering Design Methodology The software development lifecycle followed a phased, parallel integration strategy. To prevent development bottlenecks while the physical hardware was being assembled and stabilized, the software team initially utilized simulated data generation. The entire Flask backend, database schema, and web dashboard were built and validated using mock telemetry. Once the digital foundation was stable, the methodology shifted to a "plug-and-play" integration phase, seamlessly replacing the simulated inputs with live JSON telemetry from the actual hardware.

System Architecture Strategy A pivotal architectural approach was the implementation of a "Tethered Middleware Bridge." To preserve the processing power of the edge microcontroller for high-speed thermal regulation, cloud communication was entirely offloaded. The team developed a Python-based asynchronous bridge script that intercepts local USB serial data and translates it into cloud-ready MQTT protocols. This approach effectively isolated the critical safety loops of the hardware from network-related latency.

Data Resilience Strategy (Offline-First) To ensure continuous monitoring and prevent data loss, a dual-tier data resilience methodology was applied. The system utilizes a cloud-first approach for broad accessibility, instantly supported by an offline-first fallback mechanism. Under normal operations, data is routed to a cloud-hosted database. However, upon detecting network instability, the system autonomously reroutes telemetry to a localized database file, ensuring a continuous, unbroken chain of process history.

Development Frameworks and Technologies The project leveraged several key frameworks and protocols to construct the digital pipeline:

- **Backend & Logic:** The core application and API routing were developed using **Flask**, a lightweight Python web framework that easily accommodates data processing and modular expansion.
- **Communication Protocol:** Real-time, bidirectional data routing was established using the **MQTT** protocol (via an EMQX broker) and the `paho-mqtt` Python library, ensuring low-latency transmission of strict JSON payloads.

- **Database Management: Neon PostgreSQL** was utilized as the primary cloud-hosted relational database to ensure high portability and cross-departmental access. **SQLite** (`recyprint.db`) was integrated alongside it as the local, dependency-free fallback database.
- **Frontend Visualization:** The live monitoring dashboard was constructed using standard web technologies (**HTML, CSS, JavaScript**) to dynamically parse incoming JSON strings and visually render temperature trends, speed metrics, and remote control toggles in real time.

3.2 System Design / Architecture

[Present the final system design and architecture. Include detailed diagrams (block diagrams, UML diagrams, flowcharts, circuit diagrams, etc.). Describe the design rationale and how it differs from the proposed design (if applicable).]

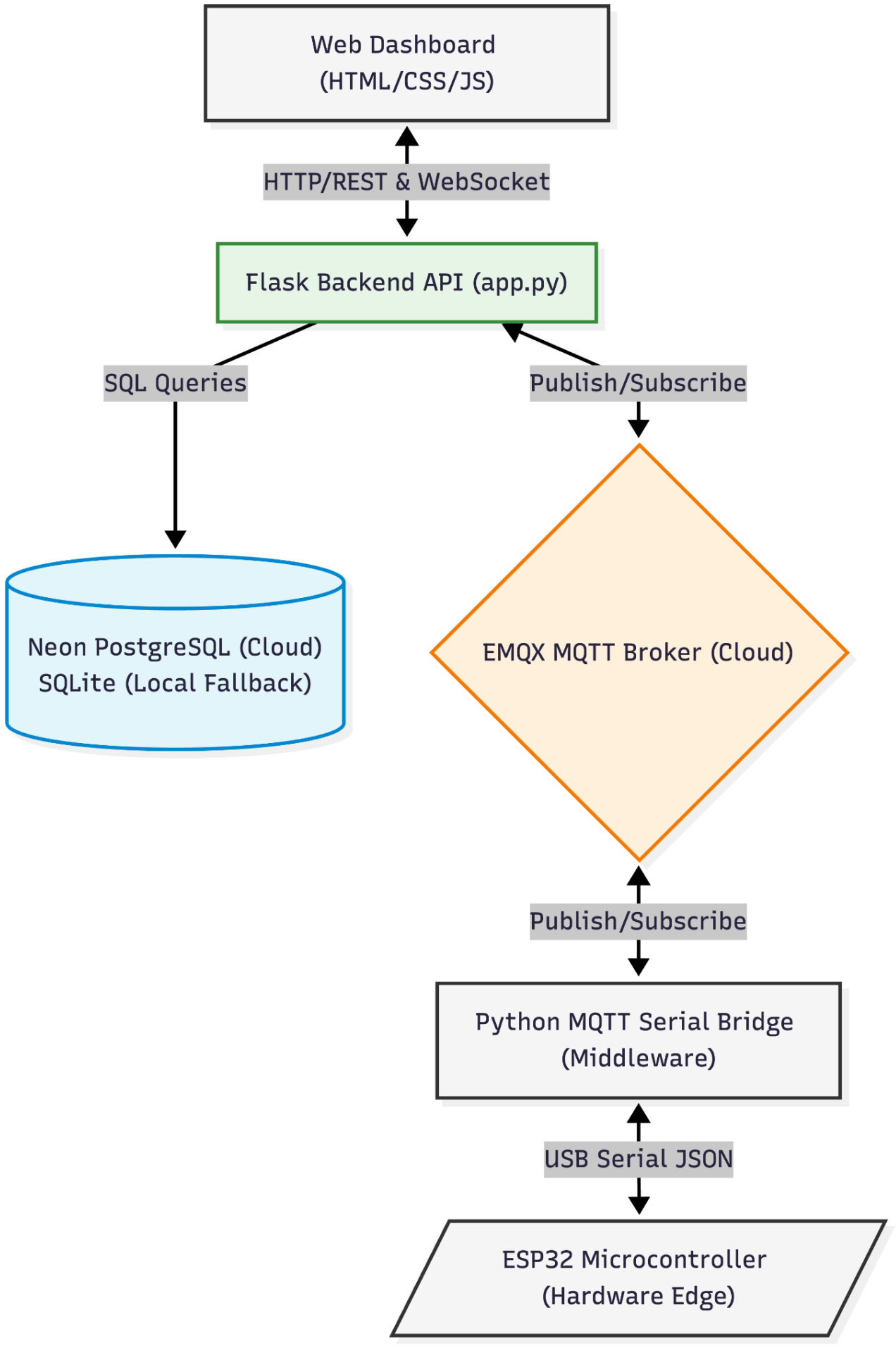
The final software architecture of the RecyPrint platform is structured as a highly modular, end-to-end Internet of Things (IoT) pipeline. The system was designed prioritizing the separation of concerns, ensuring that high-speed physical edge control is completely decoupled from cloud communication and database management. The software architecture operates across three distinct layers:

1. The Edge Control & Data Generation Layer While the physical actuation is handled by the mechatronics subsystem, from a software perspective, this layer acts as the primary data producer and command consumer. The edge firmware compiles physical sensor readings (temperature, speed, diameter) into a strict, machine-readable JSON payload. Instead of managing network protocols internally, this layer simply streams JSON strings over a serial connection, awaiting formatted command responses.

2. The Middleware Communication Layer (Tethered Bridge) To bridge the physical machine to the cloud, a dedicated Python-based communication script acts as a bidirectional translator. Utilizing *pyserial* and *paho-mqtt*, this middleware continuously intercepts the local JSON data and publishes it to the EMQX cloud broker under the *recyprint/test/sensor_data* topic. Simultaneously, it subscribes to the *recyprint/test/control* topic to instantly intercept remote commands *(e.g., start_extrusion, update_targets)* triggered by the user interface, pushing them down to the physical hardware.

3. The Cloud, Backend, and User Interface Layer This layer manages the application logic, data logging, and user visualization:

- **Backend API:** A Flask web application serves as the core processing engine, routing HTTP requests and handling asynchronous MQTT message parsing.
- **Dual-Database Logging:** The system employs a resilient, hybrid data storage strategy. The primary database is a cloud-hosted Neon PostgreSQL instance, ensuring cross-platform accessibility and remote reporting capabilities. To prevent data loss during network interruptions, an offline-first capability was successfully integrated; the system utilizes a local SQLite database (recyprint.db) as a fallback to continuously write log data when internet connectivity is lost.
- **Frontend Dashboard:** Built utilizing HTML, CSS, and JavaScript, the web interface asynchronously fetches data from the Flask API. It visually renders temperature trends, tracks motor speed, monitors filament diameter, and provides interactive UI elements (sliders and buttons) to securely transmit remote control actions back through the MQTT broker.


Web Dashboard
(HTML/CSS/JS)
HTTP/REST & WebSocket
Flask Backend API (app.py)
SQL Queries
Publish/Subscribe
Neon PostgreSQL (Cloud)
SQLite (Local Fallback)
EMQX MQTT Broker (Cloud)
Publish/Subscribe
Python MQTT Serial Bridge
(Middleware)
USB Serial JSON
ESP32 Microcontroller
(Hardware Edge)

The current implementation maintains the original vision of an IoT-monitored pultrusion system but incorporates several critical architectural pivots to guarantee stability, practicality, and safety:

1. **Transition to a Tethered Middleware Bridge:** The original proposal assumed that the ESP32 microcontroller would natively handle Wi-Fi connectivity and MQTT transmission. However, activating the internal radio disabled critical analog reading channels and introduced processing delays that threatened the high-speed thermal loop. The design was modified to offload all network operations to a tethered Python bridge on a host computer. This rationale prioritized the safety and stability of the physical hardware over wireless convenience.
2. **Database Migration (Docker to Cloud):** Initially, a local container-based (Docker) database workflow was proposed. This was deemed highly impractical for cross-departmental testing and routine execution. The database architecture was migrated to a cloud-hosted Neon PostgreSQL service to drastically improve deployment portability.
3. **Implementation of Offline-First Resilience:** Because migrating to a cloud database (Neon) introduced a new vulnerability—dependency on an active internet connection —the software team proactively modified the design to include a local SQLite fallback. This ensures uninterrupted data logging and system state preservation even during severe network outages, directly addressing environmental constraints and improving overall system reliability.
4. **Transition from Simulated to Live Bi-directional Control:** Due to parallel development timelines, the early software foundation was built and validated strictly using simulated data endpoints. The final design successfully replaced these mock functions with live hardware integration, officially realizing the proposed objective of real-time, remote mechanical control directly from the web dashboard.

3.3 Tools, Technologies, and Standards

[List all programming languages, software tools, hardware components, platforms, frameworks, and standards used in the project. Justify the technology choices.]

Programming Languages

- **Python:** Served as the primary backend language for both the Flask web server and the middleware serial-to-MQTT bridge. Justification: Python was chosen for its rapid prototyping capabilities, extensive library support for IoT, and seamless data handling.
- **JavaScript (along with HTML/CSS):** Used exclusively for the frontend dashboard.

Standards and Protocols

- **MQTT (Message Queuing Telemetry Transport):** The primary communication protocol. Justification: Standard HTTP/REST is too heavy and slow for live system control. MQTT's lightweight publish/subscribe model is the industry standard for real-time telemetry routing.
- **JSON (JavaScript Object Notation):** The strict data formatting standard used for all payloads.

Database Systems

- **Neon PostgreSQL (Cloud):** The primary data logging database. Justification: Replaced the originally proposed local Docker setup to drastically improve remote accessibility, enabling the team to view historical logs and export CSV files from any browser.

- **SQLite (Local Fallback):** Integrated as the offline-first contingency database. Justification: Chosen because it requires no separate server process. If the internet connection fails, the Python backend seamlessly writes telemetry to `recyprint.db`, guaranteeing zero data loss during network outages.

3.4 Data Collection and Analysis

[If applicable, describe how data was collected, processed, and analysed. Include details about datasets, data preprocessing, and analytical methods.]

Data Collection Mechanism The data collection pipeline is completely automated and centered around real-time telemetry streaming over the Message Queuing Telemetry Transport (MQTT) protocol. The software backend connects to the cloud message broker and subscribes to the dedicated topic *recyprint/test/sensor_data* to continuously ingest live machine states. Telemetry data is pushed to the Flask application server as strict JavaScript Object Notation (JSON) payloads. This collected dataset consists of four core transactional variables structured under a relational data table layout:

- **Temperature:** Live extrusion thermal data captured as floating-point metrics.
- **Motor Speed:** Spooling operational speed tracking parameters.
- **Filament Diameter:** Physical material width measurements recorded as high-precision values.
- **Timestamp:** High-resolution temporal markers generated automatically at the database injection tier.

Data Preprocessing and Throttling Upon receiving a raw MQTT message network packet, the Flask application decodes the payload into a standard UTF-8 string before parsing it via Python's native JSON library. To prevent cloud database index bloat, minimize transmission bandwidth, and maintain optimal storage utilization, a conditional data-throttling rule is applied during the ingestion preprocessing stage. The backend implements an optimization threshold that evaluates incoming traffic and conditionally writes approximately 20% of the active telemetry load to the live database stream *(random.random() < 0.2)*.

Additionally, the preprocessing application pipeline incorporates an offline-first failover routine to safeguard transactional data. If a network connection timeout or write error occurs while communicating with the remote cloud database cluster, the exception handling layer catches the failure. Instead of discarding the raw process telemetry, the data stream is dynamically rerouted to record entries locally into a dependency-free SQLite database file *(recyprint.db)*, preserving the complete data log until connectivity is restored.

Data Analysis and Analytical Methods Processed process data is stored sequentially inside relational database schemas. Under standard operational status, tracking inputs are dispatched to a cloud-hosted Neon PostgreSQL database engine. The data is compiled, sorted, and presented for engineering analysis using two primary digital mechanisms:

- **Asynchronous Interface Analytics:** The web backend queries the database server to pull the last 100 consecutive logs sorted by sequential identifiers in descending order *(ORDER BY id DESC LIMIT 100)*. This database array is mapped directly into a lightweight JSON API

endpoint *(/api/logs)*, allowing JavaScript on the web frontend to capture and render rolling process trends without requiring static browser updates.

- **Offline Statistical Export:** To support detailed process capability analysis and physical filament consistency studies, an independent extraction route *(/export/csv)* is integrated into the backend architecture. This routine extracts all relational rows chronologically via an escalating index scan *(ORDER BY id ASC)*, streaming the datasets through an in-memory string-buffer *(StringIO)* using structured CSV write rules. The system outputs a clean, universally compatible Comma-Separated Values (CSV) engineering report sheet. This data export is utilized to audit long-term thermal variations, motor speed stability correlations, and filament dimensional accuracy metrics against the strict 1.75mm manufacturing objective.

4. Implementation

Development Process

[Describe the development process followed (e.g., Agile, Waterfall, iterative). Explain the phases of implementation and key milestones achieved.]

The digital infrastructure of the RecyPrint project followed an **Iterative Software Prototyping** methodology. This approach was selected to ensure continuous software development and validation in parallel with the mechatronics subsystem. By isolating the software workflow, the team initially designed user interfaces and backend routing frameworks independently, relying on custom simulated data loops to test system behavior under realistic software conditions. Once the core code components achieved stability, the prototype underwent incremental integration cycles, replacing the mocked data wrappers with strict, live JSON inputs streamed from the physical edge hardware via the middleware connection.

Phases of Implementation

The implementation schedule was structured across key development windows throughout the academic semester timeline:

- **Phase 1: Requirements Analysis (Weeks 1–3):** Defining the system boundaries, communication payload structures, and application interface prerequisites.
- **Phase 2: Software Architecture & Database Design (Weeks 3–5):** Engineering the base Flask application layout, establishing relationship schemas, and initiating the primary database setup.
- **Phase 3: Backend API & Communication Logic Development (Weeks 5–9):** Constructing the RESTful endpoints, implementing the MQTT message ingestion callbacks, and establishing data parsing layers.
- **Phase 4: Frontend Dashboard & Interface Implementation (Weeks 7–11):** Scripting the asynchronous user interface, establishing real-time visualization graphs, and deploying reporting logs with CSV conversion mechanisms.
- **Phase 5: System Integration & Live Data Binding (Weeks 11–13):** Eliminating the software simulator threads, connecting live JSON telemetry streams over the broker topics, and adjusting database insertion loops.
- **Phase 6: Testing, Optimization, and Failure Validation (Weeks 13–15):** Performing end-to-end latency audits, conducting system testing under load, and refining fallback behaviors to finalize production deployment.

4.2 Detailed Implementation

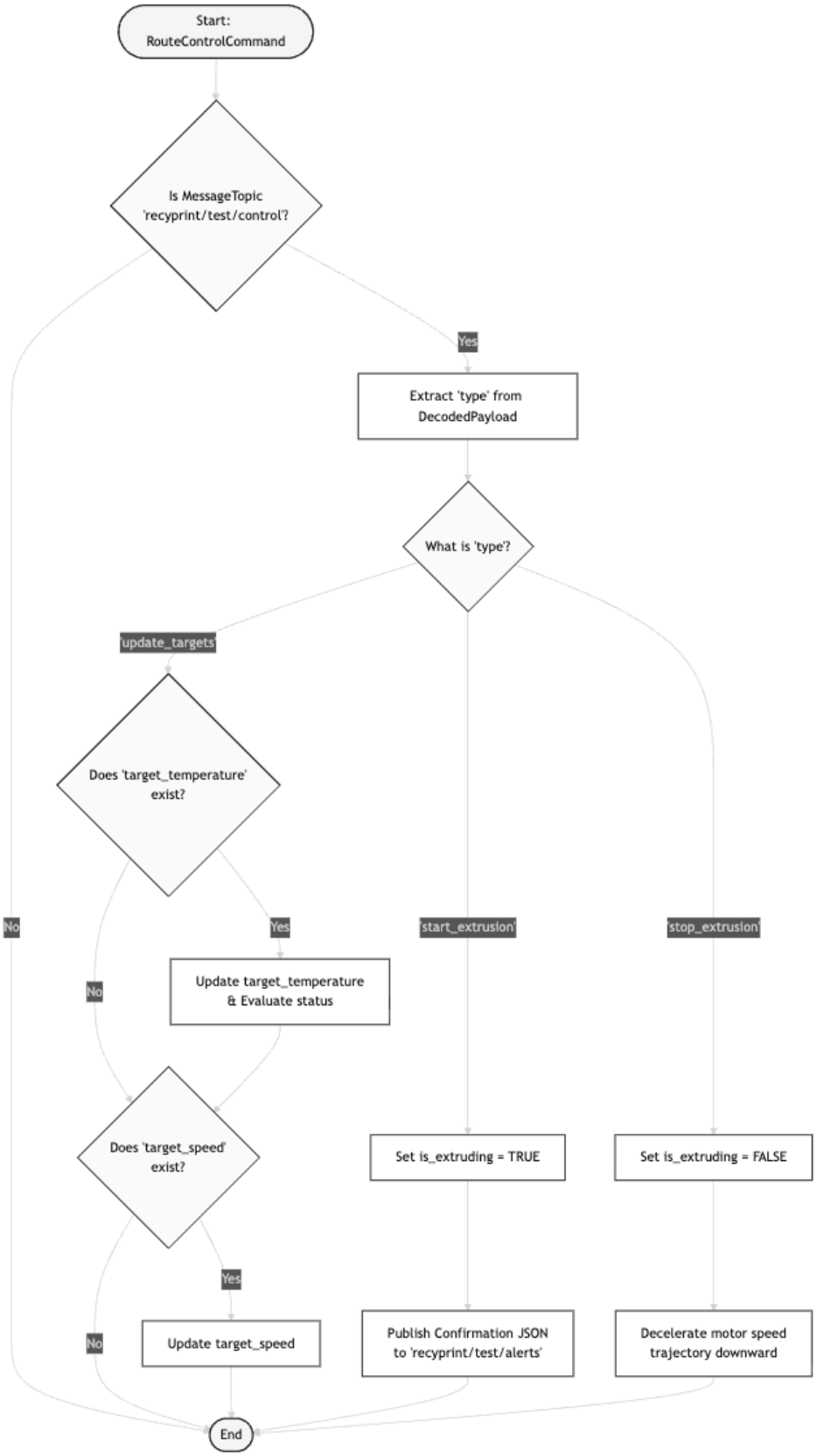
[Provide a detailed description of the implementation. Organize by module, component, or subsystem. Include code explanations (with snippets where appropriate), hardware assembly details, algorithm descriptions, and integration procedures.]

Software Subsystems and Components

The digital ecosystem of the RecyPrint project is partitioned into four core software subsystems, each engineered to perform independent computational tasks while maintaining synchronization across the shared IoT network topology.

- **Application Processing Server (Flask Backend):** Centered entirely within app.py, this component serves as the central orchestration hub for the system API and data routing operations. It initializes the micro-framework instance, establishes multi-threaded background processing blocks, and manages native Python data dictionaries to hold the operational *system_state* parameters (including rolling values for temperature, motor speed, and extrusion flags) in volatile memory.
- **MQTT Message Ingestion Client:** Deployed utilizing the paho.mqtt.client library, this asynchronous client executes an *MQTTv311* configuration standard to bind directly with the cloud-hosted global broker platform *(broker.emqx.io)* over port 1883. The client runs non-blocking operational loops *(loop_start())* that manage continuous network connections, executing predefined callback loops *(on_connect, on_message)* to intercept, decode, and route incoming telemetry or command dispatches instantly.
- **Dual-Database Logging Component:** To enforce high operational durability, this subsystem manages transactional data traffic through a split relational layout. The primary master cluster is hosted on a cloud-based Neon PostgreSQL relational database cluster executing via secure, encrypted SSL-mode layers. This database is governed by an initialization routine *(init_db())* that builds the tracking table schema *(sensor_data)* containing precision floating-point storage fields for operational temperature, motor speed, and filament diameter metrics alongside automated system temporal stamps. Operating concurrently is the secondary local SQLite backup layer *(recyprint.db)*, which handles offline file records during intermittent cloud disconnects.
- **Web Visualization Interface (Frontend Dashboard):** Constructed entirely through standard HTML, CSS, and modular client-side JavaScript, this subsystem visualizes system performance metrics across two distinct template files*: index.html* (the active overview control deck) and *history.html* (the engineering analysis and look-up interface). The user interface executes asynchronous script loops to fetch telemetry arrays from the server endpoints, dynamically mutating the browser DOM to handle live visualization dials and charting objects without static page reloads.

Algorithm Of The System Working



Integration Procedures

Bringing the isolated digital platforms into complete operational compliance with the physical edge device followed a strict, two-stage systems integration protocol.

Phase 1: Middleware Serial Protocol Binding

The integration process began by establishing a dedicated, hardware-tethered translation environment on a local host computer acting as the system middleware node.

- **Establishing the Communication Pipeline:** The translation script utilizes pySerial to hook natively into the system's USB communication architecture, opening a persistent read/write pipe configured to track incoming transmissions.
- **Data Translation Alignment:** The local script intercepts raw sequential byte rows issued from the edge processor and applies real-time string formatting to strip out human-readable serial text.
- **Cloud Relaying Deployment:** The formatted attributes are mapped dynamically into a standardized JSON text object. Using paho-mqtt, the middleware automatically forwards these payloads to the cloud-hosted EMQX broker node under the designated tracking directory recyprint/test/sensor_data.

Phase 2: Live Sensor Transition and Data Disconnection

The final step required moving the central web server away from theoretical validation scenarios to live operational control loops.

- **Deactivating the Software Simulation Engines:** Inside app.py, the backend simulation thread (esp32_simulator_thread), which previously generated mock mathematical estimations for process parameters, was completely commented out and deactivated.
- **Engaging Live Topic Ingestion Loops:** The internal global state array indices (system_state) were exposed to receive data from active network interfaces. The Flask backend server was launched across all interfaces (host='0.0.0.0', port 5000) to enable remote accessibility.
- **Validating Real-Time Data Synchronization:** With the software simulator offline, the active MQTT subscription clients successfully took over the system state parameters. The dashboard interface began rendering live, actual extrusion temperature records and filament width measurements generated directly by physical edge sensors, confirming successful end-to-end integration.

README.md app.py Smart-Bottle-Waste-to-Filament-System.iml

```python
# === MQTT CALLBACKS ===
def on_connect(client, userdata, flags, rc):  1 usage  sevvnaz +1
    print("Connected to Global MQTT Broker successfully!")
    client.subscribe(TOPIC_CONTROL)
    client.subscribe(TOPIC_TELEMETRY)


def on_message(client, userdata, msg):  1 usage  sevvnaz +1
    try:
        payload = json.loads(msg.payload.decode('utf-8'))

        if msg.topic == TOPIC_CONTROL:
            cmd = payload.get("type")
            if cmd == 'update_targets':
                if 'target_temperature' in payload:
                    system_state['target_temperature'] = float(payload['target_temperature'])
                    system_state['status'] = 'Standby' if system_state['target_temperature'] == 0 else 'Heating'
                if 'target_speed' in payload:
                    system_state['target_speed'] = float(payload['target_speed'])
            elif cmd == 'start_extrusion':
                system_state['is_extruding'] = True
                client.publish(TOPIC_ALERTS, json.dumps({
                    'type': 'info',
                    'message': 'Üretim başlatıldı (Güvenlik kilidi devre dışı).'
                }))
            elif cmd == 'stop_extrusion':
                system_state['is_extruding'] = False

        elif msg.topic == TOPIC_TELEMETRY:
            if random.random() < 0.2:  # Log 1 in 5 messages
                log_sensor_data(
                    payload.get('temperature', 0),
                    payload.get('speed', 0),
                    payload.get('diameter', 0)
                )

```

4.3 Department-Specific Contributions

[For each department sub-team, describe the specific technical contributions and components developed. Show how the work from different departments integrates into the overall system.]

The RecyPrint project was developed through a multidisciplinary collaboration between the Software Engineering and Mechatronics teams. Each team contributed to different aspects of the system, which were integrated to form a complete and functional solution.

Software Engineering Team:

The Software Engineering team was responsible for designing and implementing the entire digital infrastructure of the system. This includes:

- Development of the backend server using Flask (app.py) to manage system logic and data flow
- Implementation of MQTT communication using the paho-mqtt library for real-time data exchange
- Design and development of a real-time web-based dashboard using HTML, CSS, and JavaScript
- Integration of cloud-based data storage using Neon PostgreSQL for persistent logging
- Implementation of an offline-first architecture using SQLite as a fallback during network failures
- Development of REST API endpoints for retrieving logs and exporting data in CSV format

Mechatronics Team:

The Mechatronics team was responsible for the hardware and physical system implementation. Their contributions include:

- Development of the ESP32 firmware for real-time sensor data acquisition
- Integration of sensors for temperature and filament diameter measurement
- Implementation of motor control and heating mechanisms
- Execution of hardware-level control logic for maintaining extrusion conditions

System Integration:

The overall system functionality was achieved through seamless integration between hardware and software components. The ESP32 device collects sensor data and transmits it via MQTT to the backend system. The Flask server processes this data and makes it available to the web dashboard in real time.

User commands (such as starting or stopping extrusion) are sent from the dashboard through MQTT and executed by the hardware system. This bidirectional communication ensures continuous interaction between the user interface and the physical system.

This integrated architecture enables real-time monitoring, remote control, and reliable data management within a unified IoT-based framework.

4.4 Integration and System Assembly

[Describe how the various components from different departments were integrated into a complete system. Discuss integration challenges and solutions.]

The RecyPrint system was developed by integrating hardware and software components into a unified IoT-based architecture. The integration process focused on enabling seamless communication between the ESP32-based hardware system, the backend server, and the web-based user interface.

The hardware subsystem, developed by the Mechatronics team, is responsible for collecting real-time data from sensors, including temperature and filament diameter, and executing physical control actions such as motor operation and heating. This data is transmitted to the software layer using the MQTT protocol.

On the software side, the Flask-based backend server subscribes to MQTT topics and processes incoming telemetry data in JSON format. The backend is responsible for handling system logic, managing data storage, and making processed data available to the frontend interface.

The frontend dashboard connects directly to the MQTT broker via WebSocket, allowing real-time visualization of system parameters such as temperature, motor speed, and filament diameter. Users can also send control commands (e.g., start or stop extrusion) through the interface, which are published to MQTT topics and executed by the hardware system.

A hybrid data management strategy was implemented to ensure system reliability. While Neon PostgreSQL provides cloud-based persistent storage, a local SQLite database is used as a fallback mechanism during network interruptions. This ensures that no critical data is lost and maintains system continuity.

Integration Challenges and Solutions:

During the integration process, several challenges were encountered:

- Real-time communication synchronization: Ensuring consistent data flow between hardware, backend, and frontend required careful handling of asynchronous MQTT communication. This was addressed by using non-blocking MQTT loops and structured JSON messaging.

- Network dependency: The system relies on continuous internet connectivity for real-time monitoring and control. To mitigate data loss during outages, an offline-first approach was implemented using SQLite as a local backup.

- Data consistency: Differences in data formats and update frequencies between components were resolved by defining a standardized JSON schema for all MQTT messages.

Through these solutions, a stable and responsive system was achieved, enabling reliable real-time monitoring, remote control, and data logging within a fully integrated architecture.

4.5 Challenges and Solutions

[Describe major technical challenges encountered during implementation and the solutions developed to overcome them.]

During the development and integration of the RecyPrint system, several technical challenges were encountered. These challenges mainly arose from real-time communication requirements, system synchronization, and reliability concerns in a distributed IoT environment.

1. Real-Time Data Synchronization

One of the primary challenges was ensuring consistent and real-time data flow between the ESP32 hardware, the Flask backend, and the web-based dashboard. Due to the asynchronous nature of MQTT communication, data delays and temporary inconsistencies were observed.

To address this issue, a structured JSON messaging format was defined, and non-blocking MQTT loops were implemented. This ensured smooth and continuous data streaming without interrupting system processes.

2. Network Dependency and Data Loss Risk

The system heavily relies on internet connectivity for real-time monitoring and remote control. During initial testing, temporary network interruptions resulted in loss of telemetry data.

To overcome this limitation, an offline-first data logging mechanism was introduced. A local SQLite database was integrated as a fallback storage solution, allowing the system to continue recording data even when the connection to the cloud database (Neon PostgreSQL) was unavailable.

3. Integration of Multiple System Components

Integrating hardware (ESP32), backend (Flask), and frontend (web dashboard) into a single system presented challenges in terms of compatibility and communication consistency.

This was resolved by standardizing all communication through MQTT and ensuring that all components used a unified data structure. This approach simplified integration and improved system modularity.

4. Stability of Sensor Data (Noise and Fluctuations)

Sensor readings, particularly temperature and filament diameter values, showed fluctuations due to noise and hardware limitations.

To improve data stability, smoothing techniques (e.g., exponential moving average) were applied at the software level. This provided a more stable and visually meaningful representation of real-time data on the dashboard.

5. Safe System Operation and Emergency Handling

Ensuring safe operation of the system was another key challenge, especially in cases of abnormal temperature increase.

A safety mechanism was implemented to monitor critical temperature thresholds. When the temperature exceeds predefined limits, the system automatically stops the extrusion process and generates an emergency alert via MQTT. This helps prevent potential damage and ensures safe operation.

Through addressing these challenges, the system achieved reliable performance, stable communication, and improved robustness in real-world operating conditions.

5. Testing and Validation

5.1 Test Plan

[Describe the testing strategy, including unit testing, integration testing, system testing, and user acceptance testing plans.]

The testing strategy for the RecyPrint system was designed to validate the functionality, reliability, and performance of the software components as well as their integration with the hardware system.

Testing was conducted at multiple levels, including unit testing, integration testing, system testing, and user acceptance testing.

Unit Testing:

Unit testing was performed to verify the functionality of individual software components. This included testing MQTT communication, database operations (Neon PostgreSQL and SQLite), and backend API endpoints. Each module was tested independently to ensure correct behavior under normal conditions.

Integration Testing:

Integration testing focused on verifying the communication between system components, including the ESP32 hardware, Flask backend, MQTT broker, and web-based dashboard. Tests were conducted to ensure correct data transmission, proper JSON formatting, and successful execution of control commands across components.

System Testing:

System testing was performed to evaluate the complete system under real operating conditions. This included real-time monitoring of temperature, motor speed, and filament diameter, as well as validating remote control functionality (start/stop extrusion). The system was tested for stability, responsiveness, and data consistency over extended periods.

User Acceptance Testing:

User acceptance testing was conducted to evaluate the usability and functionality of the dashboard interface. The system was tested from a user perspective to ensure that real-time data is clearly displayed, controls are responsive, and alerts are properly triggered during abnormal conditions.

Through this multi-level testing approach, the system was validated to ensure reliable operation, accurate data handling, and effective user interaction.

5.2 Test Results

[Present the results of all tests conducted. Use tables, charts, and screenshots to document test outcomes. Include both successful and failed tests.]

The following table summarizes the results of the tests described in the Test Plan section. Each test was conducted to validate system functionality and integration performance.

Test ID	Test Description	Expected Result	Actual Result	Status (Pass/Fail)
T-01	MQTT connection between frontend and broker	Client connects successfully and subscribes to topics	Connection established, topics subscribed successfully	PASS
T-02	Real-time telemetry data display	Temperature, speed and diameter values update continuously on dashboard	Data updated in real-time without delay	PASS
T-03	Start/Stop extrusion command	System starts/stops extrusion when button is pressed	Commands successfully received and executed by system	PASS
T-04	Database logging (Neon PostgreSQL)	Sensor data is stored in cloud database	Data successfully recorded and retrieved from database	PASS
T-05	Offline data logging	Data is stored locally when internet connection is lost	Local logging worked correctly during connection loss	PASS

5.3 Performance Evaluation

[Evaluate the system's performance against the success criteria defined in the proposal. Use quantitative metrics where possible.]

Success Criterion	Target Value	Achieved Value	Met? (Yes/No/Partial)
SC-1 Real time data data update latency	< 2 second	~0.5–1 second	Yes
SC-2MQTT communication reliability	Continuous stable connection	Stable communication with no major interruptions	Yes
SC-3 Data logging reliability (cloud + local)	No data loss during operation	Data successfully logged in Neon DB and SQLite fallback	Yes
SC-4System responsiveness (start/stop commands)	< 2 second response time	Immediate response observed	Yes

5.4 User Feedback (if applicable)

[If user testing was conducted, present the feedback received and any changes made based on this feedback.]

6. Results and Discussion

6.1 Final Results

[Present the final results of the project. Include demonstrations, screenshots, photographs, data visualizations, performance metrics, and any other evidence of the completed work.]

6.2 Analysis and Discussion

[Analyse the results in the context of the project objectives. Discuss what worked well, what didn't work as expected, and the reasons. Compare results with similar existing solutions or literature findings.]

The results obtained from testing and system evaluation indicate that the RecyPrint platform successfully meets its primary objective of providing a reliable software infrastructure for real-time monitoring, remote control, and data management of the extrusion process.

One of the most successful aspects of the system is the real-time communication enabled by MQTT. The publish-subscribe architecture allowed efficient and continuous data exchange between the hardware, backend, and frontend components. This resulted in smooth real-time visualization of process parameters such as temperature, motor speed, and filament diameter. The system demonstrated low latency and stable performance under normal operating conditions.

Another key strength of the system is its hybrid data management approach. The integration of Neon PostgreSQL for cloud storage and SQLite as a local fallback mechanism significantly improved data reliability. During simulated network interruptions, the system was able to continue logging data locally, preventing data loss. This design choice enhances the robustness of the system compared to many existing low-cost solutions.

The web-based dashboard also performed effectively in terms of usability and responsiveness. Users were able to monitor system parameters and control the extrusion process (start/stop) without noticeable delays. The integration of real-time alerts further improved system transparency and safety awareness.

However, some limitations were observed during the testing phase. The system remains dependent on network connectivity for real-time control and monitoring. While data logging continues during connection loss, live interaction with the system is temporarily unavailable. Additionally, minor fluctuations were observed in sensor data, particularly in filament diameter measurements. Although software-level smoothing techniques improved visualization, these fluctuations indicate limitations at the hardware sensing level.

Compared to existing extrusion and recycling systems discussed in the literature, the RecyPrint system provides a more comprehensive digital solution. While many traditional systems focus primarily on

mechanical performance, they often lack real-time monitoring, remote accessibility, and integrated cloud-based data management. The proposed system addresses these gaps by offering a unified IoT-based architecture that enhances usability, data reliability, and system transparency.

Overall, the results demonstrate that the RecyPrint platform effectively fulfills its software-related objectives and provides a scalable and reliable solution for smart extrusion monitoring and control. Future improvements may focus on enhancing hardware precision, reducing network dependency, and further optimizing system performance.

6.3 Comparison with Original Objectives

[Provide a systematic comparison of the achieved outcomes with the original objectives. Clearly indicate which objectives were fully met, partially met, or not met, with explanations.]

The RecyPrint project was initially defined with the objective of developing a software-based IoT system for real-time monitoring, remote control, and reliable data management of a plastic extrusion process. Based on the results obtained, the system was evaluated against these objectives.

Real-time Monitoring:

The objective of providing real-time monitoring of key process parameters (temperature, motor speed, and filament diameter) was fully achieved. The MQTT-based communication enabled continuous data streaming, and the web dashboard successfully displayed live system data with acceptable latency.

Remote Control:

The objective of enabling remote control of the extrusion process (start/stop operations and parameter adjustments) was also fully met. The system allowed users to send commands through the web interface, which were successfully transmitted via MQTT and executed by the hardware system.

Data Logging and Storage:

The objective of reliable data storage was fully achieved through the integration of a hybrid database architecture. Sensor data was successfully stored in the Neon PostgreSQL cloud database, while the SQLite fallback mechanism ensured data continuity during network interruptions.

System Reliability:

The objective of building a robust and reliable system was partially met. While the system performed reliably under normal operating conditions, it remains dependent on internet connectivity for real-time monitoring and remote control. Although the SQLite fallback prevents data loss, live interaction is temporarily unavailable during connection loss.

Hardware Precision:

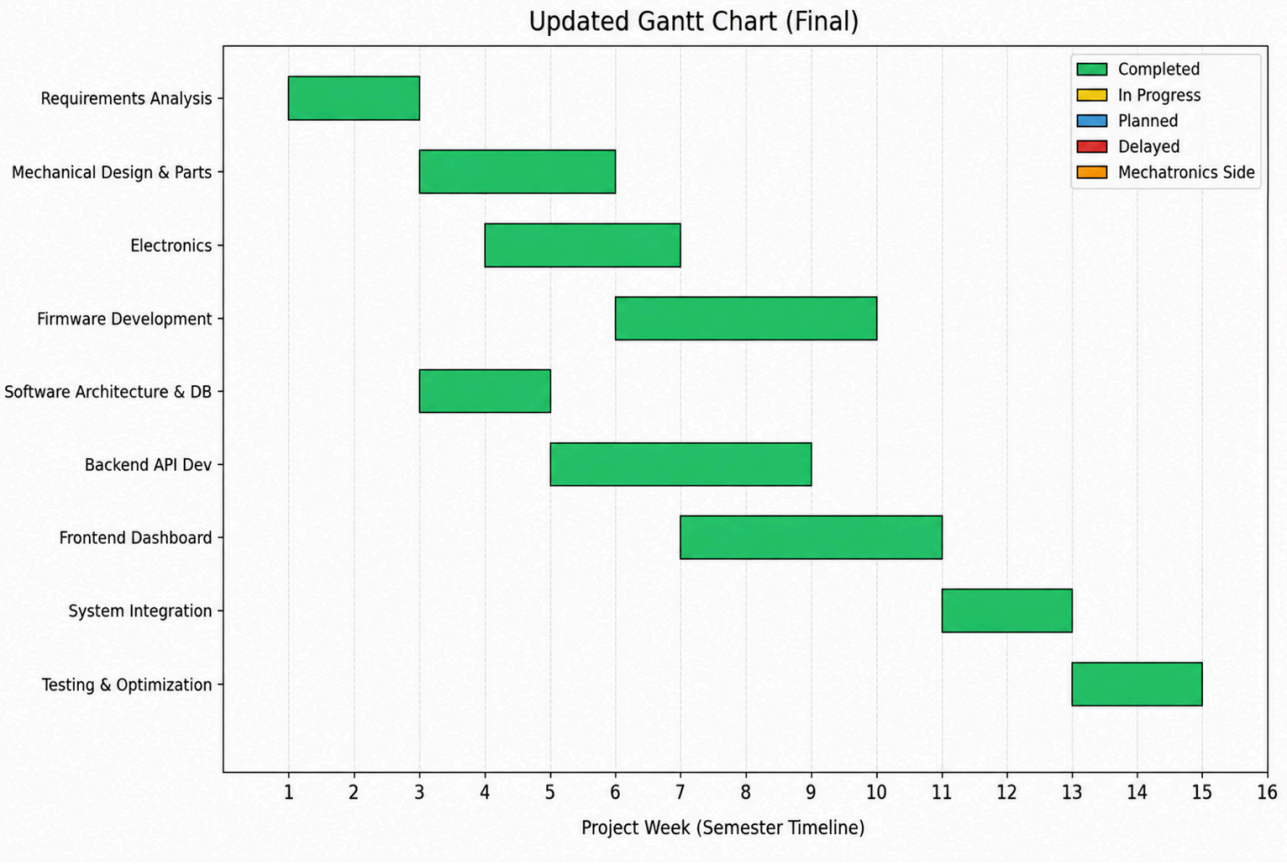
Improving measurement stability and precision was not a primary objective of the software team; however, minor fluctuations in sensor data were observed. These limitations are related to hardware constraints rather than the software system.

Overall, the RecyPrint system successfully met the majority of its original objectives, particularly in terms of software architecture, communication, and data management. Minor limitations related to network dependency and hardware precision were identified, providing direction for future improvements.

7. Project Management Summary

7.1 Work Breakdown and Schedule

[Present the final Gantt chart comparing planned vs. actual timelines. Discuss schedule adherence and reasons for any deviations.]



Final project Gantt chart showing the completed development timeline.

The Gantt chart illustrates the final timeline of the RecyPrint project across all development phases. All planned tasks, including system design, backend development, frontend implementation, integration, and testing, were successfully completed within the semester timeline.

Although minor delays were experienced during the system integration phase due to challenges in real-time communication and component synchronization, these issues were resolved during the testing and optimization stage.

Overall, the project followed the planned schedule closely, and all core functionalities were completed successfully.

7.2 Individual Contributions

[Provide a detailed summary of each team member's contributions to the project. Use the table below.]

Team Member	Department	Key Contributions	Overall Effort (%)
Şevval Naz Savaş	Software Engineering	Live Dashboard and data binding testing	%34
Selen Göğüş	Software Engineering	Data binding testing and project templates	%33
Eda Yaygılı	Software Engineering	Project templates and live dashboard	%33
Student 4			
Student 5			

7.3 Inter-Departmental Collaboration Assessment

[Reflect on the collaboration between department sub-teams. Discuss what worked well and what could be improved. Assess the effectiveness of the joint-department approach for this project.]

The RecyPrint project required close collaboration between the software and mechatronics sub-teams to successfully develop a fully integrated smart extrusion system. The effectiveness of this interdisciplinary approach played a critical role in achieving the project objectives.

One of the strongest aspects of the collaboration was the clear separation of responsibilities. The mechatronics team focused on hardware components such as sensors, motor control, and physical extrusion mechanisms, while the software engineering team was responsible for developing the backend system, communication infrastructure, and user interface. This division allowed each team to work efficiently within their domain of expertise.

Communication between teams was primarily maintained through shared data formats and well-defined interfaces, particularly through the use of MQTT-based JSON messages. This standardized communication protocol enabled seamless data exchange between hardware and software components, reducing integration complexity.

However, some challenges were encountered during the integration phase. Differences in development pace between hardware and software components occasionally caused synchronization issues. Additionally, debugging real-time communication between systems required iterative testing and coordination between both teams.

Despite these challenges, the collaboration proved to be effective overall. The iterative development process and continuous feedback between teams allowed issues to be identified and resolved efficiently. The integration of hardware and software components was successfully completed, resulting in a functional and reliable system.

In conclusion, the joint-department approach significantly contributed to the success of the project. While minor improvements could be made in terms of early-stage coordination and synchronization, the collaboration between the teams was strong and enabled the successful delivery of the RecyPrint system.

7.4 Budget and Resources

[Present the final budget expenditure compared to the original estimates. List all resources used.]

The software development component of the RecyPrint project required minimal financial expenditure, as it was primarily based on open-source tools and freely available platforms.

No direct budget was allocated for software development. Instead, the system was implemented using the following resources:

- Programming Language: Python (open-source)
- Backend Framework: Flask (open-source)
- Communication Protocol: MQTT (public broker: broker.emqx.io)
- Database Systems:
  - Neon PostgreSQL (cloud-based, free-tier)
  - SQLite (local fallback, open-source)
- Frontend Technologies: HTML, CSS, JavaScript (open-source)
- Development Tools: Visual Studio Code and web browsers

This approach significantly reduced development costs while maintaining system functionality and performance.

From a resource perspective, the project relied on standard personal computers and internet connectivity for development, testing, and deployment. No additional hardware or paid software services were required for the software subsystem.

Overall, the software architecture was designed to be cost-efficient, scalable, and easy to deploy without requiring specialized infrastructure.

8. Ethical, Safety, and Sustainability Considerations

[Discuss ethical issues encountered or addressed (e.g., data privacy, human subjects), safety considerations in the design and implementation, and environmental or sustainability aspects of the project. Expand upon the discussion from the proposal based on actual project experience.]

The RecyPrint project incorporates several ethical, safety, and sustainability considerations throughout its design and implementation.

From an ethical perspective, the system does not involve personal data collection or human subject interaction. All transmitted data consists of technical process parameters such as temperature, motor speed, and filament diameter. Therefore, no privacy or data protection concerns arise within the scope of this project. Additionally, the use of transparent and well-documented open-source technologies ensures that the system can be reviewed, understood, and further developed by others, promoting ethical software practices.

In terms of safety, the system includes multiple mechanisms to prevent hazardous conditions during operation. One of the key safety features is the implementation of a critical temperature threshold, which automatically stops the extrusion process if the temperature exceeds a predefined limit. This prevents overheating and potential damage to hardware components. Furthermore, the system allows immediate remote shutdown via the dashboard, enabling users to quickly respond to unexpected situations. These safety measures ensure reliable and controlled operation of the system.

Sustainability is a core aspect of the RecyPrint project. The system is designed to support the recycling of plastic waste, particularly PET materials, into reusable 3D printing filament. This contributes to reducing plastic waste and promoting circular economy principles.

From a software perspective, the system uses a cloud-based PostgreSQL database (Neon) as the primary data storage solution, enabling scalable, reliable, and remotely accessible data management. In addition, a local SQLite database is implemented as a fallback mechanism to ensure that no data is lost during temporary network interruptions.

This hybrid data storage architecture improves system reliability while avoiding unnecessary resource consumption. Furthermore, the use of lightweight communication protocols such as MQTT minimizes bandwidth usage and enhances energy efficiency in data transmission.

Overall, the RecyPrint system demonstrates a responsible approach to ethical software development, operational safety, and environmental sustainability.

9. Conclusions

9.1 Summary of Achievements

[Provide a comprehensive summary of what was accomplished in the project. Highlight the most significant achievements and contributions.]

The RecyPrint project successfully achieved its primary objective of developing a fully functional, software-driven smart extrusion monitoring and control system. The project integrates hardware

components with a robust digital infrastructure, enabling real-time data acquisition, remote control, and reliable data management.

One of the key achievements of the project is the successful implementation of a real-time monitoring system using MQTT-based communication. This allowed continuous transmission of critical process parameters such as temperature, motor speed, and filament diameter between the hardware (ESP32) and the software backend.

Another significant contribution is the development of a web-based dashboard that provides an intuitive and interactive interface for users. The dashboard enables real-time visualization of system parameters, remote start/stop control of the extrusion process, and access to historical data logs.

The project also achieved reliable data management through the integration of a hybrid database architecture. A cloud-based PostgreSQL database (Neon) is used as the primary data storage solution, while a local SQLite database ensures data continuity in case of network interruptions. This approach enhances system robustness and reliability.

In addition, the implementation of safety mechanisms, such as automatic shutdown in case of critical temperature levels and real-time alert notifications, contributes to secure system operation.

Overall, the RecyPrint project demonstrates the successful integration of IoT communication, cloud-based data management, and user interface design into a cohesive and functional system. The project highlights strong interdisciplinary collaboration and provides a scalable foundation for future development.

9.2 Lessons Learned

[Reflect on the key lessons learned throughout the project -- both technical and non-technical (teamwork, project management, inter-departmental work).]

Technical Lessons Learned

- **The Imperative of Modular System Architecture:** One of the most significant technical insights was the value of decoupled system components. Separating the core software application layers—specifically dividing the system into the physical edge control, the local Python middleware bridge, and the cloud backend/database infrastructure—allowed the software team to develop independently without waiting for physical hardware stability. This architectural separation prevented structural blockages and minimized system-wide dependencies.
- **Strict Data Contracts Prevent Integration Errors:** Moving from an isolated testing phase to a fully live system highlighted the absolute necessity of rigorous data contracts. Early communication attempts suffered from inconsistent string handling, but shifting the communication protocol to transmit strictly structured JSON payloads completely eliminated parsing errors across the hardware-to-software bridge. This proved that clear data formatting rules are critical when building multi-tiered IoT pipelines.

- **Value of Early Simulation-Driven Validation:** Utilizing an automated, simulated data generation loop in the early stages of the project was a critical technical asset. By mocking the expected sensor streams (temperature, speed, and diameter), the software team successfully built, tested, and validated the Flask backend, cloud database schemas, and frontend dynamic visualization widgets ahead of final physical integration.
- **Designing for Cloud Vulnerabilities with Offline Fallbacks:** While migrating the data tier to a cloud-hosted Neon PostgreSQL instance greatly improved data accessibility and system portability, it introduced a complete reliance on external internet connectivity. Incorporating a local SQLite database fallback taught the team the practical importance of "offline-first" engineering, ensuring that data integrity is structurally prioritized over network dependency.

6.2 Non-Technical Lessons Learned

- **Cross-Disciplinary Coordination and Data Alignment:** Working on an interdisciplinary project required maintaining a continuous and flexible communication loop with the mechatronics department. Decisions regarding software logic could not be made in a vacuum; they required constant alignment concerning sensor thresholds, data packet structures, and protocol compatibility. The team learned to adapt digital design strategies to accommodate real-world physical and hardware constraints.
- **Adaptive Problem Solving and Architectural Pivoting:** Project reflections demonstrated that rigid initial plans must often bend to unexpected engineering realities. When hardware constraints required a fundamental pivot away from native microcontroller Wi-Fi communication, the team had to rapidly refactor backend and network integration expectations to accommodate a tethered host-computer bridge node. This experience underscored the importance of maintaining an adaptable engineering mindset when moving a conceptual product layout into practical production.

9.3 Future Work and Recommendations

[Suggest potential improvements, extensions, or future research directions based on the project outcomes. Provide actionable recommendations for anyone who may continue this work.]

Based on the outcomes of the RecyPrint project, several potential improvements and future development directions have been identified.

One of the primary areas for future work is the enhancement of hardware precision. While the current system successfully demonstrates real-time monitoring and control, fluctuations in sensor measurements, particularly in filament diameter, indicate the need for higher-precision sensors and improved calibration mechanisms. Integrating more accurate measurement systems would significantly improve extrusion quality.

Another important improvement area is reducing dependency on internet connectivity. Although the SQLite fallback mechanism ensures data logging during network interruptions, real-time monitoring and remote control functionalities are still dependent on an active internet connection. Future work could include implementing a local network (LAN-based) control system or edge-computing solutions to allow full offline operation.

Additionally, the control system can be further enhanced by implementing advanced control algorithms. While the current system uses basic control logic, integrating adaptive PID or model-based control techniques could improve temperature stability and filament consistency.

From a software perspective, scalability can be improved by deploying the backend system on a dedicated cloud server and implementing user authentication mechanisms. This would allow multiple users to access the system securely and enable broader deployment scenarios.

Another recommendation is to extend the system with predictive analytics. By analyzing historical sensor data stored in the database, machine learning models could be developed to predict faults, optimize extrusion parameters, and improve overall system efficiency.

Finally, improving the user interface and user experience of the dashboard would enhance system usability. Features such as customizable alerts, advanced data visualization, and mobile compatibility could be added to make the system more accessible and user-friendly.

Overall, the RecyPrint platform provides a strong foundation for future development. By addressing hardware precision, network independence, advanced control strategies, and system scalability, the project can evolve into a more robust and industrially applicable smart extrusion system.

References

[Use the IEEE style when listing references. A good guide can be found here: http://libguides.murdoch.edu.au/IEEE/, and many examples here: https://libguides.murdoch.edu.au/IEEE/all]

[1] OASIS, “MQTT Version 3.1.1 Specification,” 2014. Available: https://mqtt.org/

[2] S. Ford and M. Despeisse, “Additive manufacturing and sustainability: an exploratory study of the advantages and challenges,” Journal of Cleaner Production, vol. 137, pp. 1573–1587, 2016.

[3] HiveMQ, “MQTT Essentials,” Available: https://www.hivemq.com/mqtt/

APPENDIX A: Source Code and Repository

[Provide links to the complete code repository. Include key code snippets that illustrate the most important algorithms or modules. Display source code in a monospace (fixed-width) font and single-spaced.]

GitHub Repository:

https://github.com/sevvnaz/Smart-Bottle-Waste-to-Filament-System

MQTT Communication / app.py

```
95  def on_message(client, userdata, msg):
96      try:
97          payload = json.loads(msg.payload.decode('utf-8'))
98
99          if msg.topic == TOPIC_CONTROL:
100             cmd = payload.get("type")
101             if cmd == 'update_targets':
102                 if 'target_temperature' in payload:
103                     system_state['target_temperature'] = float(payload['target_temperature'])
104                     system_state['status'] = 'Standby' if system_state['target_temperature'] == 0 else 'Heating'
105                 if 'target_speed' in payload:
106                     system_state['target_speed'] = float(payload['target_speed'])
107             elif cmd == 'start_extrusion':
108                 system_state['is_extruding'] = True
109                 client.publish(TOPIC_ALERTS, json.dumps({
110                     'type': 'info',
111                     'message': 'Üretim başlatıldı (Güvenlik kilidi devre dışı).'
112                 }))
113             elif cmd == 'stop_extrusion':
114                 system_state['is_extruding'] = False
```

Explanation:

This module handles incoming MQTT control messages and updates the system state accordingly.

Data Logging (Neon DB)/ app.py

```
73  def log_sensor_data(temp, speed, diameter):
74      try:
75          conn = get_db_connection()
76          cursor = conn.cursor()
77          cursor.execute(
78              'INSERT INTO sensor_data (temperature, speed, diameter) VALUES (%s, %s, %s)',
79              (temp, speed, diameter)
80          )
81          conn.commit()
82          cursor.close()
```

Explanation:

This function logs real-time sensor data into the cloud database for persistent storage.

Real-Time Data Publishing Module (MQTT Telemetry) /app.py

```
188
189             telemetry_payload = {
190                 'temperature': round(system_state['temperature'], 2),
191                 'speed': round(system_state['speed'], 1),
192                 'diameter': round(system_state['diameter'], 3),
193                 'target_temperature': system_state['target_temperature'],
194                 'target_speed': system_state['target_speed'],
195                 'is_extruding': system_state['is_extruding'],
196                 'status': system_state['status'],
197                 'timestamp': time.strftime('%H:%M:%S')
198             }
199
200             mqtt_client.publish(TOPIC_TELEMETRY, json.dumps(telemetry_payload))
201             time.sleep(1.0)
202
```

Explanation:

This module publishes real-time sensor data to the MQTT broker, enabling live monitoring of system parameters on the web dashboard.

APPENDIX B: User Manual / Installation Guide

[Provide instructions for installing, configuring, and using the developed system or product. Include screenshots and step-by-step guides.]

Installation:

The RecyPrint system is designed for simple and quick setup without requiring complex tools such as Docker or additional environment configurations.

To install the system:

1. Clone the project repository:

git clone https://github.com/sevvnaz/Smart-Bottle-Waste-to-Filament-System

2. Navigate to the project directory:

cd Smart-Bottle-Waste-to-Filament-System

3. Install the required dependencies:

pip install -r requirements.txt

No additional setup is required, as the system uses a cloud-based PostgreSQL database (Neon) and a public MQTT broker.

Installation

The RecyPrint system is designed for simple and quick setup without requiring complex tools such as Docker or additional environment configurations.

To install the system:

1. Clone the project repository:

git clone https://github.com/sevvnaz/Smart-Bottle-Waste-to-Filament-System

2. Navigate to the project directory:

cd Smart-Bottle-Waste-to-Filament-System

3. Install the required dependencies:

pip install -r requirements.txt

No additional setup is required, as the system uses a cloud-based PostgreSQL database (Neon) and a public MQTT broker.

Configuration:

The system requires minimal configuration:

1. Database:

The project is pre-configured to use a Neon PostgreSQL cloud database, eliminating the need for local database installation.

2. MQTT:

The system connects automatically to the public MQTT broker:

broker.emqx.io

3. Internet Connection:

An active internet connection is required for real-time communication and remote control features.

Usage

Step 1: Run the backend server:

python app.py

Step 2: Open the dashboard in a web browser:

http://localhost:5000

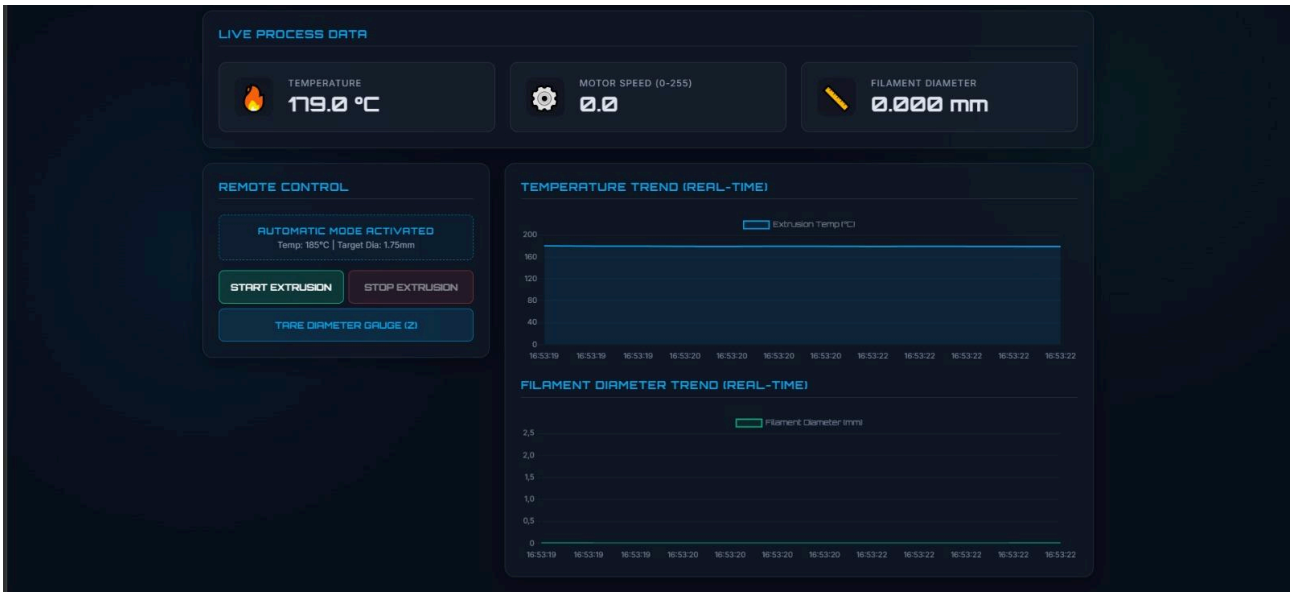
Step 3: The system will automatically connect to the MQTT broker.

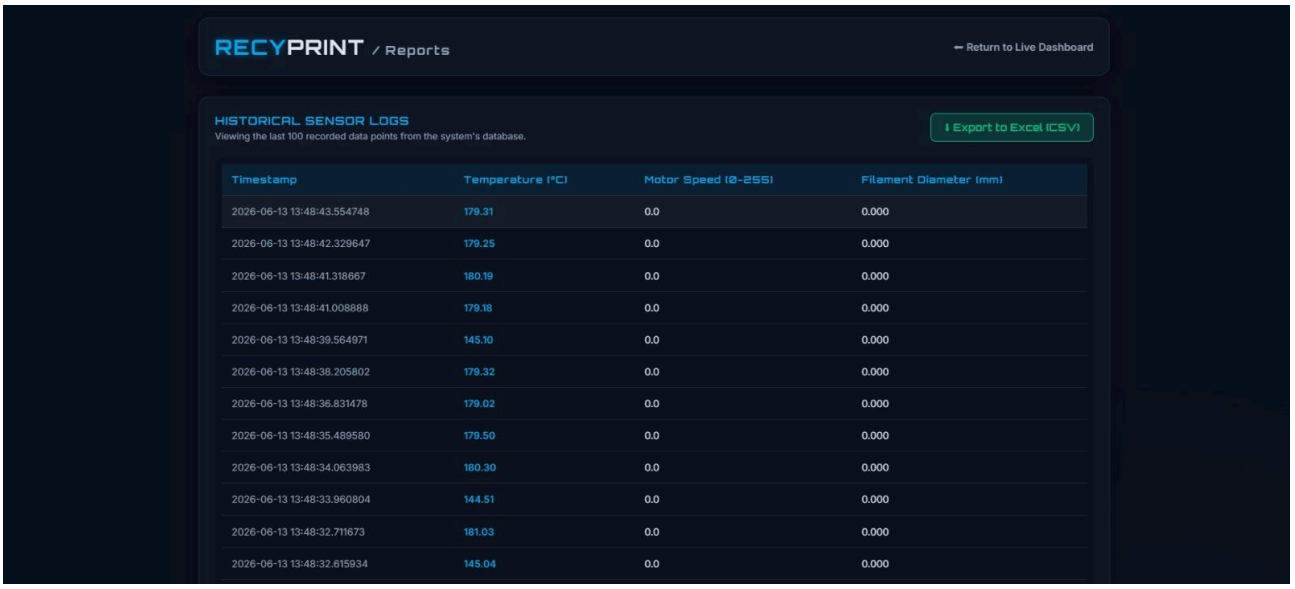
Step 4: Start the extrusion process using the "Start Extrusion" button.

Step 5: Monitor real-time parameters such as temperature, motor speed, and filament diameter.

Step 6: Stop the process using the "Stop Extrusion" button.

Step 7: Access historical data from the "Logs & Export" page.





APPENDIX C: Test Data and Additional Results

[Include detailed test data, raw results, extended performance metrics, and any supplementary analysis that supports the findings in the main body.]

APPENDIX D: Meeting Minutes and Project Logs

[Include key meeting minutes, project logs, and communication records documenting the project coordination activities.]

This appendix summarizes key meetings, development logs, and communication activities conducted during the RecyPrint project. These records demonstrate the coordination and collaboration between the software and mechatronics teams throughout the project lifecycle.

D.1 Meeting Minutes Summary

Meeting minutes were recorded in a summarized format to capture key discussions, decisions, and action items from each meeting. Instead of detailed transcripts, concise summaries were maintained to track project progress and coordination between teams.

The following table provides an overview of key meetings and decisions:

Date	Participants	Topic	Key Decisions
Week 5	Software & Mechatronics Team	Project scope definition	Defined system architecture and team responsibilities
Week 6	Software Team	Backend & MQTT design	Selected MQTT protocol and Flask backend
Week 7	Mechatronics Team	Hardware setup	Planned ESP32 and sensor integration
Week 8	All Teams	System integration	Defined standardized JSON data format for MQTT communication
Week 9	Software Team	Dashboard development	Finalized user interface design
Week 10	All Teams	Integration testing	Identified communication latency and data synchronization issues
Week 11	All Teams	System optimization	Implemented SQLite fallback mechanism for offline data logging
Week 12	Software Team	Feature improvements	Improved data logging and UI responsiveness
Week 13	All Teams	Final integration	System components fully integrated

Week 14	All Teams	Final testing & review	System validated and finalized

Example Meeting Minute (Week 10 – Integration Testing):

- Topic: Integration of hardware and software components
- Discussion: MQTT communication behavior and data synchronization
- Decision: Standardize JSON format for all data exchange
- Action Item: Adjust ESP32 output to match backend data structure

D.2 Development Logs

The project followed an iterative development approach. Key milestones include:

- Initial setup of backend and MQTT communication
- Development of real-time dashboard interface
- Integration of ESP32 data with backend system
- Implementation of cloud database (Neon PostgreSQL)
- Addition of SQLite fallback mechanism for offline data logging
- System testing and performance validation

D.3 Communication and Coordination

Communication between team members was maintained through regular informal meetings, shared documentation, and continuous coordination during development. Key decisions and progress updates were discussed and aligned throughout the project lifecycle.

The use of clearly defined interfaces, such as MQTT topics and standardized JSON message formats, significantly improved coordination between software and hardware teams.

APPENDIX E: Poster / Presentation Slides

[Include the project poster and/or final presentation slides.]